

1. Chatbot 和 Agent 的区别

Chatbot 面向的是“问答”，Agent 面向的是“行动”。

Chatbot 的典型流程是：

用户输入 → 模型理解 → 生成回答

Agent 的典型流程是：

用户目标 → 任务拆解 → 选择工具 → 执行动作 → 观察结果 → 调整计划 → 输出结果

这两者最大的区别在于：Chatbot 主要在语言空间里工作，Agent 则进入了真实系统。它可能读取文件、查询数据库、调用 API、生成报告、发起审批、运行代码、写入业务系统。也正因为如此，Agent 的工程问题远比 Chatbot 复杂。Chatbot 答错了，主要是内容质量问题；Agent 做错了，可能会造成数据错误、流程错误、权限泄露甚至真实业务损失。

2. Tool Calling 的五步流程

Tool Calling 是 Agent 从“只会说”变成“能做事”的关键机制。一个标准流程可以拆成五步。

第一步，**理解任务**。模型先判断用户真正要完成什么，而不只是复述问题。比如用户说“帮我看一下这周市场有什么异动”，背后任务不是闲聊，而是要提取市场数据、识别异常、解释原因并生成摘要。

第二步，**选择工具**。模型判断是否需要调用工具，以及调用哪个工具。比如查行情数据、检索新闻、读取内部指标、生成图表等。

第三步，**生成参数**。模型按照 tool schema 生成结构化参数，例如市场、日期范围、指标类型、top_k 数量等。

第四步，**执行工具并返回结果**。系统校验参数、检查权限，然后执行工具，把结构化结果返回给模型。

第五步，**基于结果继续推理或输出**。模型读取工具结果，判断任务是否完成。如果不够，就继续调用工具；如果已经足够，就生成最终答案。

所以 Tool Calling 不是简单的“模型调 API”，而是 Agent 和外部世界之间的工程接口。

3. ReAct 的核心思想

ReAct 可以理解为 **Reasoning + Acting**，也就是“边思考，边行动”。它的核心不是让模型一次性想完所有步骤，而是让模型形成一个循环：

Reason: 我现在知道什么？下一步需要什么？**Act:** 调用哪个工具？**Observe:** 工具返回了什么？
Reason: 根据新结果，是否需要继续？

这个机制很重要，因为真实任务往往不是一步完成的。比如市场异动分析中，Agent 先发现“港股科技股大涨”，但原因不清楚，于是继续检索新闻；检索后发现可能与政策、业绩或流动性有关，再进一步判断哪个证据更强。ReAct 的价值在于，它让 Agent 不再是一次性生成答案，而是在外部反馈中逐步逼近任务结果。

4. Agent 为什么需要状态管理

Agent 必须有状态管理，因为任务执行不是单轮问答。它需要记住：用户目标是什么，已经调用过哪些工具，拿到了哪些事实，哪些假设被验证，哪些步骤失败，当前任务进度到哪里。

没有状态管理，Agent 很容易出现三类问题：重复调用同一个工具、忘记前面已经得到的结果、无法判断任务是否完成。比如一个报告生成 Agent，如果不知道自己已经完成了“数据读取”和“异常识别”，就可能反复查数据；如果忘记某个工具失败过，就可能陷入无限重试。

状态管理的本质，是把 Agent 从“聊天模型”变成“任务流程系统”。它不只是保存对话历史，更要保存任务状态、工具状态、证据状态、错误状态和完成状态。

5. 为什么多 Agent 不一定更好

多 Agent 听起来很高级，但不一定更好。很多任务并不需要多个 Agent，只需要一个 Agent 加上清晰的工具、状态机和权限控制。

多 Agent 的问题在于：通信成本高、责任边界复杂、结果容易冲突、调试难度上升。比如一个“研究 Agent”、一个“数据 Agent”、一个“写作 Agent”、一个“审查 Agent”同时工作，如果没有清晰的任务契约，最后可能出现互相等待、重复劳动、结论不一致的问题。

更稳妥的思路是：先把单 Agent 做扎实，再考虑多 Agent。只有当任务天然存在强分工，例如数据处理、合规审查、报告撰写、人工审批明显分离时，多 Agent 才可能带来收益。否则，多 Agent 可能只是把一个问题拆成多个更难调试的问题。

6. Agent 的失败模式

Agent 常见失败模式包括以下几类。

第一，**无限循环**：反复调用工具，却没有新信息。

第二，**工具误用**：该查数据库时去搜索网页，该读文件时调用生成工具。

第三，**参数错误**：日期、范围、ID、枚举值填错。

第四，**幻觉补全**：工具没有返回结果，模型却编造原因。

第五，**权限越界**：访问了不该访问的数据，或执行了不该执行的动作。

第六，**状态丢失**：忘记已经完成的步骤，导致任务混乱。

第七，**结果不可验证**：输出看起来完整，但没有证据来源和可追溯链路。

这些失败模式说明，Agent 的难点不是“模型聪不聪明”，而是系统有没有边界、校验、状态、日志和降级。

7. 如何设计一个“市场异动分析 Agent”

把“市场异动分析 Agent”设计成三层：Data 层、Agent 层和 Report 层。

Data 层负责确定性计算，不交给模型。它每天读取市场数据，包括指数、行业、个股、利率、汇率、商品、成交额、波动率等指标，然后计算涨跌幅、分位数、历史均值、robust z-score、成交放量、跨资产联动等指标，生成异常候选清单。这里的原则是：凡是能用代码稳定计算的，就不要让模型猜。

Agent 层负责解释和判断。它拿到异常候选后，先识别哪些异动值得分析，再调用工具检索新闻、公告、宏观事件、政策信息和历史相似情形。它的输出不能直接写成确定因果，而要区分事实、推断和不确定性。例如：

事实：恒生科技指数本周上涨 X%，处于近一年涨幅分位数前 5%。

证据：市场数据、新闻标题、政策文件。

推断：可能与流动性改善、政策预期或业绩修复有关。

不确定性：尚无法确认单一主因。

Report 层负责把结果渲染成固定格式的周报或卡片，例如“异动事实—幅度—可能驱动—影响—风险提示—证据来源”。最终报告必须带引用、带时间戳、带数据口径说明。

这个 Agent 的关键不是让模型“自由分析市场”，而是让它在一套受控流程里工作：

数据计算 → 异常识别 → 证据检索 → 原因解释 → 置信度标注 → 报告生成 → 人工复核

其中，模型负责综合解释，代码负责计算，工具负责取证，权限系统负责边界，审计系统负责追踪。这样的市场异动分析 Agent，才不是一个会聊天的财经助手，而是一个可以进入真实工作流的金融任务执行系统。